JAVA PROJECT

1. Voting System (using HashMap)

Concepts Used

- HashMap (Candidate → Vote Count)
- Loops & Conditions
- OOP (Class: Candidate)
- File I/O (store results)
- Exception Handling

Features to Add

- Add candidates
- Cast vote
- Prevent duplicate voting
- Display results
- Winner announcement
- Save results to file

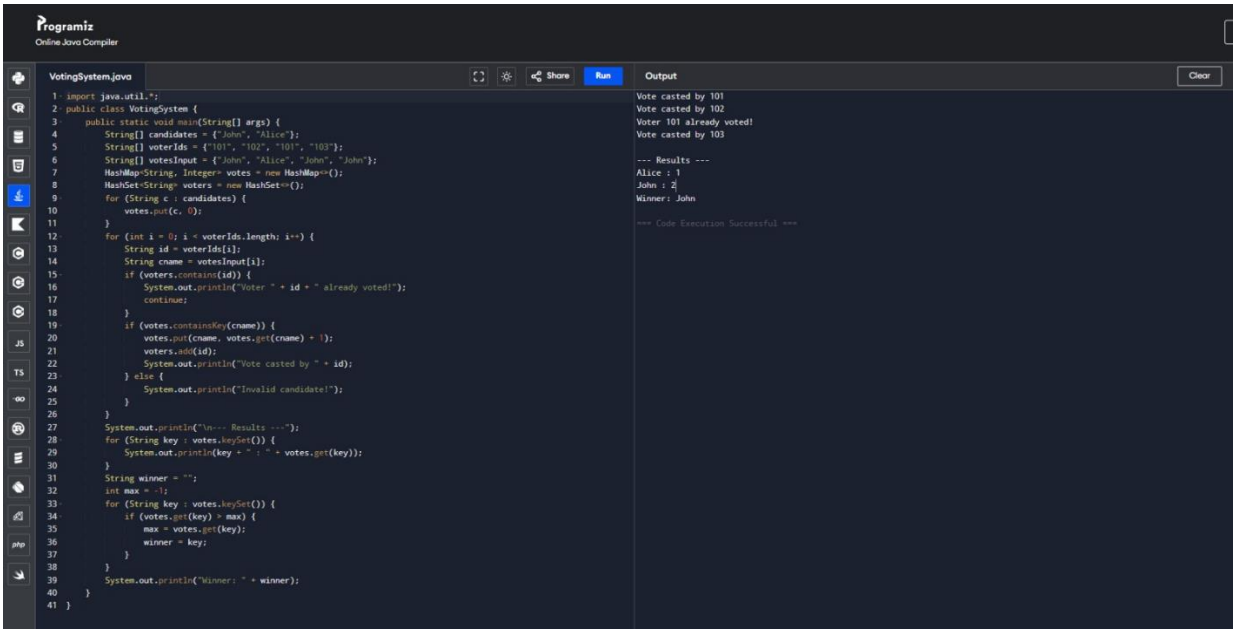
Code: import java.util.*;

```
import java.io.*;
class Account {
    int accNo;
    String name;
    double balance;
    ArrayList<String> history = new ArrayList<>();
    Account(int accNo, String name, double balance) {
        this.accNo = accNo;
```

```
        this.name = name;
        this.balance = balance;
        history.add("Account created with balance: " + balance);
    }
    void deposit(double amt) {
        balance += amt;
        history.add("Deposited: " + amt);
    }
    void withdraw(double amt) {
        if (amt <= balance) {
            balance -= amt;
            history.add("Withdrawn: " + amt);
        } else {
            history.add("Withdraw failed (Insufficient balance)");
        }
    }
}
public class BankSystem {
    public static void main(String[] args) {
        HashMap<Integer, Account> bank = new HashMap<>();
        bank.put(101, new Account(101, "Nirosha", 5000));
        bank.put(102, new Account(102, "Arun", 3000));
        bank.get(101).deposit(2000);
        bank.get(101).withdraw(1000);
```

```
    bank.get(102).withdraw(5000); // fail case
    System.out.println("--- Balance ---");
    for (Account a : bank.values()) {
        System.out.println(a.name + " : " + a.balance);
    }
    System.out.println("\n--- Transaction History ---");
    for (Account a : bank.values()) {
        System.out.println(a.name + ":");
        for (String h : a.history) {
            System.out.println("  " + h);
        }
    }
    try {
        BufferedWriter bw = new BufferedWriter(new FileWriter("bank.txt"));
        for (Account a : bank.values()) {
            bw.write(a.accNo + " " + a.name + " " + a.balance);
            bw.newLine();
        }
        bw.close();
        System.out.println("\nData saved to file");
    } catch (Exception e) {
        System.out.println("File error");
    }
}
```

}



---

3. Log Analyzer System

Concepts Used

- File I/O (read logs)
- BufferedReader
- String methods (split, contains)
- Collections (HashMap)

Features to Add

- Read log file
- Count errors/warnings
- Filter logs by keyword
- Display summary report
- Search logs by date/type

Code: import java.util.*;

```
public class LogAnalyzer {
    private List<String> logs = new ArrayList<>();
    private Map<String, Integer> logCount = new HashMap<>();
    public void loadSampleLogs() {
        logs.clear();
        logCount.clear();
        logs.add("2025-04-01 INFO Application started");
        logs.add("2025-04-01 WARNING Low memory");
        logs.add("2025-04-01 ERROR Database connection failed");
        logs.add("2025-04-02 INFO User login successful");
        logs.add("2025-04-02 ERROR Null pointer exception");
        logs.add("2025-04-03 WARNING Disk space low");
        for (String log : logs) {
            analyzeLine(log);
      }
        System.out.println("Sample logs loaded successfully!");
    }
    private void analyzeLine(String line) {
        String lower = line.toLowerCase();
        if (lower.contains("error")) {
            logCount.put("ERROR", logCount.getOrDefault("ERROR", 0) + 1);
        } else if (lower.contains("warning")) {
            logCount.put("WARNING", logCount.getOrDefault("WARNING", 0) + 1);
```

```
    } else if (lower.contains("info")) {
        logCount.put("INFO", logCount.getOrDefault("INFO", 0) + 1);
    }
}
public void filterByKeyword(String keyword) {
    System.out.println("\nFiltered Logs:\n");
    for (String log : logs) {
        if (log.toLowerCase().contains(keyword.toLowerCase())) {
            System.out.println(log);
        }
    }
}
public void displaySummary() {
    System.out.println("\n===== LOG SUMMARY =====");
    System.out.println("ERROR  : " + logCount.getOrDefault("ERROR", 0));
    System.out.println("WARNING: " + logCount.getOrDefault("WARNING", 0));
    System.out.println("INFO   : " + logCount.getOrDefault("INFO", 0));
}
public void searchByType(String type) {
    System.out.println("\nLogs of type: " + type);
    for (String log : logs) {
        if (log.toLowerCase().contains(type.toLowerCase())) {
            System.out.println(log);
        }
```

```
    }
}
public void searchByDate(String date) {
    System.out.println("\nLogs on date: " + date);
    for (String log : logs) {
        if (log.contains(date)) {
            System.out.println(log);
        }
    }
}
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    LogAnalyzer analyzer = new LogAnalyzer();
    while (true) {
        System.out.println("\n===== LOG ANALYZER =====");
        System.out.println("1. Load Sample Logs");
        System.out.println("2. Filter by Keyword");
        System.out.println("3. Show Summary");
        System.out.println("4. Search by Type");
        System.out.println("5. Search by Date");
        System.out.println("6. Exit");
        System.out.print("Enter choice: ");
        int choice = Integer.parseInt(sc.nextLine());
```

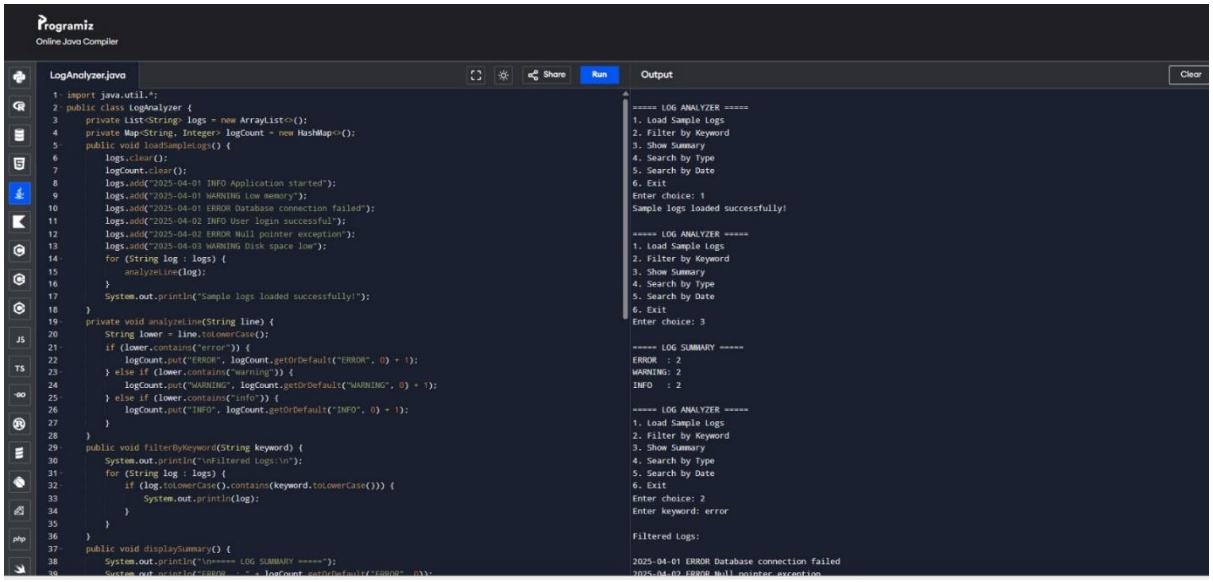
```
switch (choice) {
    case 1:
        analyzer.loadSampleLogs();
        break;
    case 2:
        System.out.print("Enter keyword: ");
        analyzer.filterByKeyword(sc.nextLine());
        break;
    case 3:
        analyzer.displaySummary();
        break;
    case 4:
        System.out.print("Enter type (ERROR/WARNING/INFO): ");
        analyzer.searchByType(sc.nextLine());
        break;
    case 5:
        System.out.print("Enter date (YYYY-MM-DD): ");
        analyzer.searchByDate(sc.nextLine());
        break;
    case 6:
        System.out.println("Exiting...");
        return;
    default:
        System.out.println("Invalid choice!");
```

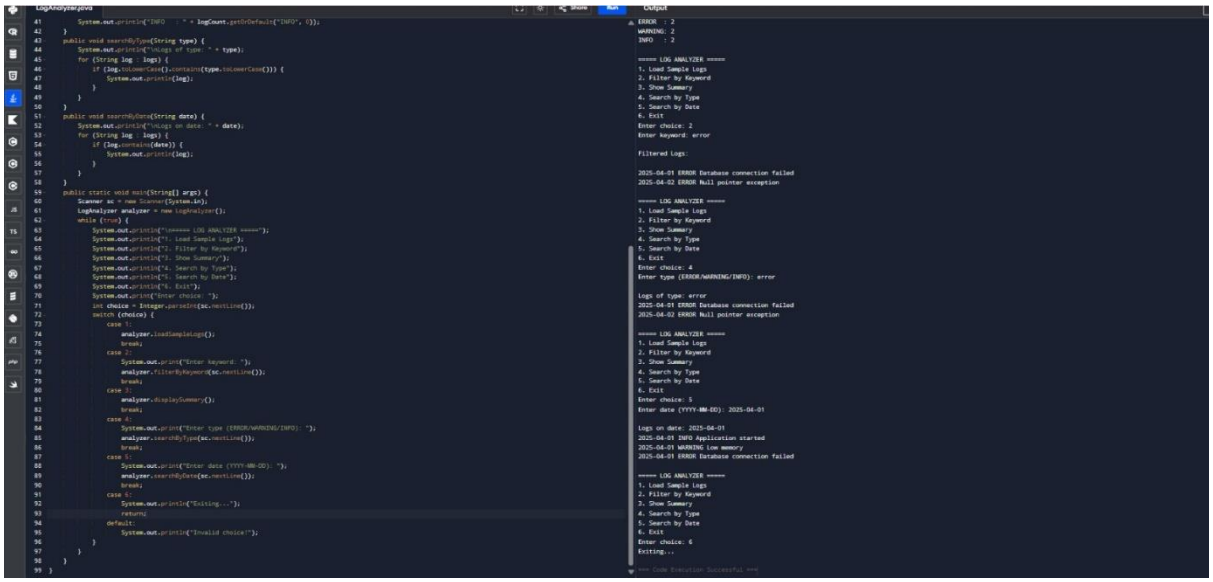
```
        }

    }

  }

}
```





---

4. Text-Based Search Engine

Concepts Used

- File Handling

- ArrayList / HashMap
- String processing
- Searching algorithms

Features to Add

- Load text files
- Search keyword
- Show matching lines
- Count word frequency
- Highlight results (basic)
- Transaction history
- Save data to file

Code: import java.io.*;

```
import java.util.*;

public class TextSearchEngine {

    private List<String> lines; // store file lines

    private Map<String, Integer> wordFrequency; // word count

    public TextSearchEngine() {

        lines = new ArrayList<>();

        wordFrequency = new HashMap<>();

    }

    public void loadFile(String filePath) {

        try {

            BufferedReader br = new BufferedReader(new FileReader(filePath));

            String line;
```

```
        while ((line = br.readLine()) != null) {
            lines.add(line);
            processWords(line);
        }
        br.close();
        System.out.println("File loaded successfully!\n");
    } catch (IOException e) {
        System.out.println("Error loading file: " + e.getMessage());
    }
}
private void processWords(String line) {
    String[] words = line.toLowerCase().split("\\W+");
    for (String word : words) {
        if (!word.isEmpty()) {
            wordFrequency.put(word, wordFrequency.getOrDefault(word, 0) + 1);
        }
    }
}
public void searchKeyword(String keyword) {
    keyword = keyword.toLowerCase();
    boolean found = false;
    System.out.println("\nMatching lines:\n");
    for (String line : lines) {
        if (line.toLowerCase().contains(keyword)) {
```

```
                found = true;
                String highlighted = line.replaceAll("(?i)" + keyword, "[" +
keyword.toUpperCase() + "]");
                System.out.println(highlighted);
            }
        }
        if (!found) {
            System.out.println("No matches found.");
        }
    }
    public void showWordFrequency(String word) {
        word = word.toLowerCase();
        int count = wordFrequency.getOrDefault(word, 0);

System.out.println("Word '" + word + "' appears " + count + " times.");
    }
    public void showTopWords() {
        System.out.println("\nTop frequent words:\n");
        wordFrequency.entrySet()
                .stream()
                .sorted((a, b) -> b.getValue() - a.getValue())
                .limit(10)
                .forEach(entry ->
                        System.out.println(entry.getKey() + " : " + entry.getValue())
                );
```

```
}
public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    TextSearchEngine engine = new TextSearchEngine();
    while (true) {
        System.out.println("\n===== TEXT SEARCH ENGINE =====");
        System.out.println("1. Load File");
        System.out.println("2. Search Keyword");
        System.out.println("3. Word Frequency");
        System.out.println("4. Top Words");
        System.out.println("5. Exit");
        System.out.print("Enter choice: ");
        int choice = scanner.nextInt();
        scanner.nextLine();
        switch (choice) {
            case 1:
                System.out.print("Enter file path: ");
                String path = scanner.nextLine();
                engine.loadFile(path);
                break;
            case 2:
                System.out.print("Enter keyword: ");
                String keyword = scanner.nextLine();
                engine.searchKeyword(keyword);
```

```
                break;
            case 3:
                System.out.print("Enter word: ");
                String word = scanner.nextLine();
                engine.showWordFrequency(word);
                break;
            case 4:
                engine.showTopWords();
                break;
            case 5:
                System.out.println("Exiting...");
                scanner.close();
                return;
            default:
                System.out.println("Invalid choice!");
        }
    }
  }
}
```

Programiz
Online Java Compiler

TextSearchEngine.java

Share | Run

```java
import java.io.*;
import java.util.*;
public class TextSearchEngine {
    private List<String> lines;
    private Map<String, Integer> wordFrequency;
    public TextSearchEngine() {
        lines = new ArrayList<>();
        wordFrequency = new HashMap<>();
    }
    public void loadFile(String filePath) {
        try {
            BufferedReader br = new BufferedReader(new FileReader(filePath));
            String line;
            while ((line = br.readLine()) != null) {
                lines.add(line);
                processWords(line);
            }
            br.close();
            System.out.println("File loaded successfully!\n");
        } catch (IOException e) {
            System.out.println("Error loading file: " + e.getMessage());
        }
    }
    private void processWords(String line) {
        String[] words = line.toLowerCase().split("\\W+");
        for (String word : words) {
            if (!word.isEmpty()) {
                wordFrequency.put(word, wordFrequency.getOrDefault(word, 0) + 1);
            }
        }
    }
    public void searchKeyword(String keyword) {
        keyword = keyword.toLowerCase();
        boolean found = false;
        System.out.println("\nMatching lines:\n");
        for (String line : lines) {
            if (line.toLowerCase().contains(keyword)) {
                found = true;
                String highlighted = line.replaceAll("(?i)" + keyword, "[" + keyword.toUpperCase() + "]");
                System.out.println(highlighted);
            }
        }
        if (!found) {
            System.out.println("No matches found.");
        }
    }
    public void showWordFrequency(String word) {
        word = word.toLowerCase();
        int count = wordFrequency.getOrDefault(word, 0);
        System.out.println("Word '" + word + "' appears " + count + " times.");
    }
    public void showTopWords() {
        System.out.println("\nTop frequent words:\n");
        wordFrequency.entrySet()
                .stream()
                .sorted((a, b) -> b.getValue() - a.getValue())
                .limit(10)
                .forEach(entry ->
                        System.out.println(entry.getKey() + " : " + entry.getValue())
```

Output | Clear

```
===== TEXT SEARCH ENGINE =====
1. Load File
2. Search Keyword
3. Word Frequency
4. Top Words
5. Exit
Enter choice: 3
Enter word: word
Word 'word' appears 0 times.

===== TEXT SEARCH ENGINE =====
1. Load File
2. Search Keyword
3. Word Frequency
4. Top Words
5. Exit
Enter choice: 4

Top frequent words:


===== TEXT SEARCH ENGINE =====
1. Load File
2. Search Keyword
3. Word Frequency
4. Top Words
5. Exit
Enter choice: 5
Exiting...

=== Code Execution Successful ===
```